

This is an exercise set, to teach the Regex, NER (Named Entity Extraction), and PDF extraction handling. Understand the concepts by question/answer with Chatgpt, understand each exercises goal, what is expected to be done and build code as per the suggested directory structure. Refer to the starter code section, only when necessary.

Teaching Strategy (Important)

Each level teaches a key concept:

Level	Concept
1–2	Text + Regex
3	NER
4	Hybrid systems
5	Normalization
6	Fuzzy matching
7	Context
8–9	Real-world noise
10	Production system

LEVEL 1 — Text Basics (Foundation)

Exercise 1: Read Text from File

🎯 Learn basic file handling

📋 Task:

- Read a `.txt` file containing a medical report
- Print line count, word count

📁 Input:

- `sample_report.txt`

✓ Output:

```
{  
  "lines": 120,  
  "words": 1450  
}
```

Exercise 2: Clean OCR Noise

🔗 Handle real-world messy text

🛠️ Task:

- Remove:
 - Extra spaces
 - Broken words (Jo hn)
 - Random line breaks

📁 Input:

Dr. Jo hn Sm ith
MD

✓ Output:

Dr. John Smith MD

● LEVEL 2 — Regex Extraction

Exercise 3: Extract Doctor Names (Regex Only)

🔗 Introduce pattern matching

🛠️ Task:

- Extract names like:
 - Dr. John Smith
 - John Smith, MD

📁 Input:

Seen by Dr. John Smith. Report signed by Jane Doe, MD.

✓ Output:

```
[  
  "Dr. John Smith",  
  "Jane Doe, MD"  
]
```

Exercise 4: Handle Variations

🔗 Increase regex complexity

📋 Task:

Support:

- SMITH, JOHN
- J. Smith
- Dr John A Smith MD

📁 Input:

Consulted with SMITH, JOHN and J. Smith.

Exercise 5: Extract Multiple PII Types

🔗 Broaden scope

📋 Task:

Extract:

- Names
- Dates
- Phone numbers

📁 Input:

Patient John Smith visited on 12/05/2024. Contact: 9876543210

LEVEL 3 — NER Introduction

Exercise 6: Use spaCy for PERSON Extraction

🔗 Learn NER basics

📌 Task:

- Extract all PERSON entities

📄 Input:

Patient was seen by Dr. John Smith and Nurse Jane Doe.

✓ Output:

```
["John Smith", "Jane Doe"]
```

Exercise 7: Compare Regex vs NER

🎯 Understand tradeoffs

📌 Task:

- Run both regex + NER
- Compare results:
 - Which missed what?

📄 Output:

```
{  
  "regex_only": [...],  
  "ner_only": [...],  
  "common": [...]  
}
```

● LEVEL 4 — Hybrid Extraction (Real World)

Exercise 8: Merge Regex + NER Results

🎯 Build production logic

📌 Task:

- Combine outputs
- Remove duplicates using span overlap

📁 Input:
Same text with overlapping matches

Exercise 9: Add Confidence Scores

🔗 Introduce ranking

📁 Task:

- Regex → 0.95
 - NER → 0.85
 - Both → 0.98
-

● LEVEL 5 — Name Normalization

Exercise 10: Normalize Names

🔗 Canonical formatting

📁 Task:

Convert:

- SMITH, JOHN → John Smith
- J. Smith → J Smith
- DR. JOHN SMITH MD → John Smith

📁 Output:

```
{  
  "original": "SMITH, JOHN",  
  "normalized": "John Smith"  
}
```

Exercise 11: Split Name Components

🔗 Structured data

🏰 Task:

Extract:

```
{
  "first": "John",
  "last": "Smith",
  "prefix": "Dr",
  "suffix": "MD"
}
```

● LEVEL 6 — Fuzzy Matching

Exercise 12: Match Similar Names

🔗 Handle variations

🏰 Task:

Use RapidFuzz or similar:

- "John Smith" vs "J Smith"

Return similarity score

Exercise 13: Group Same Entities

🔗 Clustering

🏰 Task:

Group:

```
John Smith
J Smith
SMITH, JOHN
```

✓ Output:

```
{
  "entity_1": [
    "John Smith",
    "J Smith",
    "SMITH, JOHN"
  ]
}
```

● LEVEL 7 — Context Awareness

Exercise 14: Add Role Detection

🔗 Contextual enrichment

📋 Task:
Identify:

- Doctor
- Nurse
- Patient

📄 Input:

Dr. John Smith examined patient Mary Jane. Nurse Alex assisted.

Exercise 15: Disambiguate Same Names

🔗 Real-world complexity

📋 Task:

Dr John Smith (Radiology)
Dr John Smith (Emergency)

→ Should they be same?

○ LEVEL 8 — PDF Processing

Exercise 16: Extract Text from PDF

🔗 Entry into PDF parsing

📋 Task:
Use:

- pdfplumber or
- PyMuPDF

📁 Input:

- `medical_report.pdf`
-

Exercise 17: Handle Multi-page Documents

🔗 Real pipeline

🔧 Task:

- Extract text page-wise
 - Preserve page numbers
-



LEVEL 9 — OCR Challenges

Exercise 18: Handle OCR Errors

🔗 Real-world robustness

🔧 Task:

Fix:

- "J0hn Sm1th"
 - "Dr . John"
-

Exercise 19: Line Merge Heuristics

🔗 Preprocessing

🔧 Task:

Merge:

Dr. John
Smith MD

LEVEL 10 — Mini Project (Production Simulation)

Exercise 20: Build Full Pipeline

 End-to-end system

 Task:
Pipeline:

PDF → Clean → Regex + NER → Normalize → Cluster

 Input:

- 2–3 medical PDFs

 Output:

```
{
  "entities": [
    {
      "canonical_name": "Dr. John Smith",
      "variants": [
        "Dr John Smith",
        "SMITH, JOHN",
        "J Smith MD"
      ],
      "role": "doctor"
    }
  ]
}
```

Recommended Input Dataset

You should prepare:

1. Clean text

- sample_report.txt

2. Noisy OCR text

- ocr_report.txt

3. PDFs

- 2–5 ED medical reports

4. Ground truth (important!)

```
{  
  "expected_entities": [...]  
}
```

1. REPO STRUCTURE

```
pii-extraction-training/  
├── README.md  
├── requirements.txt  
├── data/  
│   ├── raw/  
│   │   ├── sample_report.txt  
│   │   ├── ocr_report.txt  
│   │   ├── medical_report_1.txt  
│   │   └── medical_report_2.txt  
│   └── expected/  
│       ├── level3_regex.json  
│       ├── level6_ner.json  
│       └── final_entities.json  
├── exercises/  
│   ├── level1/  
│   ├── level2/  
│   ├── level3/  
│   ├── level4/  
│   ├── level5/  
│   ├── level6/  
│   ├── level7/  
│   ├── level8/  
│   ├── level9/  
│   └── level10/  
├── utils/  
│   ├── text_cleaning.py  
│   └── regex_patterns.py
```

```
|
| | ner_model.py
| | normalization.py
| | fuzzy_match.py
|
| api/
| | app.py
```

2. REQUIREMENTS

```
spacy
rapidfuzz
pdfplumber
pymupdf
fastapi
uvicorn
```

3. SAMPLE DATASETS

data/raw/sample_report.txt

```
Patient was evaluated by Dr. John A. Smith, MD.
Follow-up scheduled with Jane Doe, MD.
```

data/raw/ocr_report.txt

```
Dr. John Smith
MD examined the patient.
Seen by NURSE JANE DOE RN
```

data/raw/medical_report_1.txt

```
Emergency Department Report
```

```
Patient: Michael Brown
Attending Physician: Dr. John Smith MD
Nurse: Jane Doe RN
```

```
Radiology reviewed by SMITH, JOHN.
Follow-up with J. Smith.
```

data/expected/final_entities.json

```
{
  "entities": [
    {
      "canonical_name": "Dr John Smith",
      "variants": [
        "Dr. John Smith MD",
        "SMITH, JOHN",
        "J. Smith"
      ],
      "role": "doctor"
    }
  ]
}
```

4. STARTER CODE (BY LEVEL)

LEVEL 1 — File + Cleaning

exercises/level1/read_file.py

```
def read_file(path):
    with open(path, "r") as f:
        text = f.read()
    return text

if __name__ == "__main__":
    text = read_file("../..data/raw/sample_report.txt")
    print(len(text.split()))
```

utils/text_cleaning.py

```
import re

def clean_text(text):
    text = re.sub(r"\s+", " ", text)
    text = re.sub(r"(\w)\s+(\w)", r"\1\2", text) # fix broken words
    return text.strip()
```

LEVEL 2 — Regex

utils/regex_patterns.py

```
import re

PATTERNS = [
    r"\bDr\.\?\s+[A-Z][a-z]+\s+[A-Z][a-z]+\b",
    r"\b[A-Z][a-z]+\s+[A-Z][a-z]+\s*MD\b",
    r"\b[A-Z]{2,}\s*[A-Z]{2,}\b",
    r"\b[A-Z]\.\s+[A-Z][a-z]+\b"
]

compiled_patterns = [re.compile(p) for p in PATTERNS]

def extract_names_regex(text):
    results = []
    for pattern in compiled_patterns:
```

```
    for match in pattern.finditer(text):
        results.append(match.group())
    return list(set(results))
```

exercises/level2/extract_regex.py

```
from utils.regex_patterns import extract_names_regex

text = open("../..data/raw/medical_report_1.txt").read()
print(extract_names_regex(text))
```

LEVEL 3 — NER

utils/ner_model.py

```
import spacy

nlp = spacy.load("en_core_web_sm")

def extract_names_ner(text):
    doc = nlp(text)
    return [ent.text for ent in doc.ents if ent.label_ == "PERSON"]
```

exercises/level3/extract_ner.py

```
from utils.ner_model import extract_names_ner

text = open("../..data/raw/medical_report_1.txt").read()
print(extract_names_ner(text))
```

LEVEL 4 — Merge

exercises/level4/merge.py

```
from utils.regex_patterns import extract_names_regex
from utils.ner_model import extract_names_ner

def merge(regex_list, ner_list):
    return list(set(regex_list + ner_list))

text = open("../..data/raw/medical_report_1.txt").read()

r = extract_names_regex(text)
```

```
n = extract_names_ner(text)

print(merge(r, n))
```

● LEVEL 5 — Normalization

utils/normalization.py

```
import re

def normalize_name(name):
    name = name.lower()
    name = re.sub(r"\b(dr|md|rn)\b", "", name)

    if "," in name:
        last, first = name.split(",")
        name = first.strip() + " " + last.strip()

    name = re.sub(r"^[^w\s]", "", name)
    return " ".join(name.split()).title()
```

exercises/level5/normalize.py

```
from utils.normalization import normalize_name

names = ["SMITH, JOHN", "Dr. John Smith MD"]

for n in names:
    print(normalize_name(n))
```

● LEVEL 6 — Fuzzy Matching

utils/fuzzy_match.py

```
from rapidfuzz import fuzz

def similarity(a, b):
    return fuzz.token_set_ratio(a, b)
```

exercises/level6/group.py

```
from utils.fuzzy_match import similarity
```

```
names = ["John Smith", "J Smith", "SMITH, JOHN"]

groups = []

for name in names:
    placed = False
    for g in groups:
        if similarity(name, g[0]) > 90:
            g.append(name)
            placed = True
            break
    if not placed:
        groups.append([name])

print(groups)
```

● LEVEL 7 — Role Detection

exercises/level7/roles.py

```
def detect_role(text):
    text = text.lower()
    if "dr" in text or "md" in text:
        return "doctor"
    if "nurse" in text or "rn" in text:
        return "nurse"
    return "unknown"
```

LEVEL 8 — PDF (Simulated)

exercises/level8/pdf_extract.py

```
import pdfplumber

def extract_pdf(path):
    text = ""
    with pdfplumber.open(path) as pdf:
        for page in pdf.pages:
            text += page.extract_text() + "\n"
    return text
```

LEVEL 9 — OCR Cleanup

exercises/level9/ocr_fix.py


```
def fix_ocr(text):
    text = text.replace("0", "o")
    text = text.replace("1", "i")
    return text
```

LEVEL 10 — FULL PIPELINE

exercises/level10/pipeline.py

```
from utils.text_cleaning import clean_text
from utils.regex_patterns import extract_names_regex
from utils.ner_model import extract_names_ner
from utils.normalization import normalize_name
from utils.fuzzy_match import similarity

text = open("../data/raw/medical_report_1.txt").read()

text = clean_text(text)

names = list(set(
    extract_names_regex(text) +
    extract_names_ner(text)
))

normalized = [normalize_name(n) for n in names]

# simple clustering
clusters = []
for name in normalized:
    found = False
    for c in clusters:
        if similarity(name, c[0]) > 90:
            c.append(name)
            found = True
            break
    if not found:
        clusters.append([name])

print(clusters)
```
